

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA

**ARTHUR ERPEN
CAETANO PERUZZO
GABRIEL HERRERA**

Relatório sobre a Etapa 4 de Sistemas Operacionais II

Florianópolis
2026

Resumo

Este relatório descreve a implementação da Etapa 4 do projeto de Sistemas Operacionais II: a adição de *coordenadas espaciais* às mensagens e a sincronização espacial entre sistemas autônomos com precisão de quadrante, realizada por meio de um **módulo de kernel próprio** (`gps.ko`) que expõe a posição da VM via `ioctl` sobre o arquivo de dispositivo `/dev/gps`. Cada `Message` ganha um campo `quadrant` que viaja no cabeçalho do `Protocol`; a NIC carimba o quadrante de origem em cada envio e descarta na recepção mensagens vindas de quadrantes distintos. As coordenadas são inicialmente aleatórias e simulam um deslocamento com direção variável (passo entre quadrantes vizinhos, sorteado a cada ≥ 3 s por avanço preguiçoso baseado em `jiffies`). O texto apresenta os requisitos, a arquitetura, os detalhes de implementação e a validação experimental pelo teste `quadrant` (9 VMs: 4 RSUs ancoradas uma em cada quadrante + 5 veículos móveis), executado em QEMU sobre kernel próprio PREEMPT_RT 6.15.5.

Sumário

1	Introdução	2
2	Módulo de kernel GPS	2
2.1	Interface <code>ioctl</code> e dispositivo <code>/dev/gps</code>	2
2.2	Estado inicial aleatório	3
2.3	Deslocamento simulado: avanço preguiçoso e direção variável	3
2.4	Operação atômica e wrapper de <i>userspace</i>	4
3	Etiquetagem espacial das mensagens	4
3.1	Quadrante no Header do <code>Protocol</code>	4
3.2	Etiqueta na <code>Message</code> (API)	5
4	Integração com a NIC: carimbo no envio e filtragem na recepção	6
4.1	Visão geral	6
4.2	Acessores estáticos e injeção na NIC	6
4.3	Carimbo no envio	6
4.4	Filtragem na recepção	7
4.5	Recepção na aplicação: <code>Protocol::receive</code> e <code>Communicator</code>	7
5	Compartilhamento intra-veículo	8
6	Validação experimental	8
7	Resultados	9
8	Conclusão	10

1 Introdução

A Etapa 3 acrescentou ao sistema a dimensão temporal: cada mensagem passou a carregar um *timestamp* único e os relógios das VMs foram sincronizados via SPTP. A Etapa 4 acrescenta a dimensão *espacial*: cada mensagem passa a carregar a coordenada do emissor (um quadrante), e os sistemas autônomos sincronizam suas posições relativas com a precisão necessária para identificar em qual quadrante cada VM se encontra. A filtragem espacial é aplicada na NIC, de modo que sistemas em quadrantes distintos não trocam mensagens entre si.

Requisitos da etapa (enunciado):

- Mensagens (**Message**) devem ser *etiquetadas com coordenadas espaciais na origem*.
- Sistemas autônomos devem sincronizar suas posições com precisão suficiente para situá-los em quadrantes específicos.
- Componentes do mesmo sistema autônomo *compartilham* a percepção do espaço e concordam exatamente sobre sua localização.
- A sincronização espacial inter-sistemas deve ser feita por um *módulo de kernel* com interface que retorne as coordenadas de cada VM.
- As coordenadas são inicialmente *aleatórias*, e em seguida simulam um *deslocamento com velocidade variável*.
- Formato das mensagens: $M = \{origin, timestamp, payload\}$, $PTP = \{address/origin, timestamp,$

O formato PTP da Etapa 3 foi preservado; o trabalho desta etapa concentra-se em (i) construir o módulo de kernel GPS, (ii) propagar o quadrante de origem como parte do **Header** do protocolo, e (iii) filtrar mensagens cross-quadrante na NIC.

2 Módulo de kernel GPS

2.1 Interface ioctl e dispositivo /dev/gps

A sincronização espacial é provida por um módulo de kernel próprio, `kernel/gps_module/gps.ko`, registrado como *misc device* (`MISC_MAJOR = 10`, `MISC_DYNAMIC_MINOR`) e exposto ao *userspace* como `/dev/gps`. O módulo não expõe `read` nem `write`: a única interface é via duas chamadas `ioctl` declaradas em `gps_ioctl.h`:

```
#define GPS_IOC_MAGIC      'g'
#define GPS_IOC_GET_QUADRANT _IOR(GPS_IOC_MAGIC, 1, int) // userspace l
#define GPS_IOC_SET_FIXED   _IO (GPS_IOC_MAGIC, 2)      // s comando
#define GPS_QUADRANTS      4
```

O `file_operations` do módulo expõe apenas `unlocked_ioctl`:

```
static const struct file_operations gps_fops = {
    .owner          = THIS_MODULE,
    .unlocked_ioctl = gps_ioctl,
};
static struct miscdevice gps_misc = {
```

```
.minor = MISC_DYNAMIC_MINOR,
.name  = "gps",
.fops  = &gps_fops,
};
```

Ao carregar (`insmod`), `misc_register` cria `/dev/gps` automaticamente via `devtmpfs`. A interface satisfaz o requisito de “*módulo de kernel com uma interface que retorne as coordenadas de cada VM*”.

2.2 Estado inicial aleatório

A coordenada inicial é definida no momento do `insmod`: por padrão, sorteia-se um quadrante via `get_random_u32() % 4`. Um parâmetro opcional `initial_quadrant` permite forçar uma posição inicial — usado no teste para ancorar cada RSU em um quadrante diferente:

```
static int initial_quadrant = -1; /* -1 = aleatorio */
module_param(initial_quadrant, int, 0444);

static int __init gps_init(void) {
    if (initial_quadrant >= 0 && initial_quadrant < GPS_QUADRANTS)
        quadrant = initial_quadrant;
    else
        quadrant = get_random_u32() % GPS_QUADRANTS;
    last_change = jiffies;
    ...
}
```

2.3 Deslocamento simulado: avanço preguiçoso e direção variável

O deslocamento é simulado sem threads nem timers, por meio de um *avanço preguiçoso* disparado dentro do próprio ioctl `GET_QUADRANT`: a cada leitura, o módulo verifica via `jiffies` se o intervalo mínimo (`3 * HZ`, 3s) já expirou; em caso afirmativo, sorteia o próximo quadrante e atualiza `last_change`. A escolha do próximo quadrante é restrita aos *vizinhos imediatos* na grade 2×2 :

Q0	Q1
Q2	Q3

A tabela de vizinhança é codificada explicitamente:

```
static const int gps_neighbors[GPS_QUADRANTS][2] = {
    {1, 2}, /* Q0: vizinhos Q1, Q2 */
    {0, 3}, /* Q1: vizinhos Q0, Q3 */
    {0, 3}, /* Q2: vizinhos Q0, Q3 */
    {1, 2}, /* Q3: vizinhos Q1, Q2 */
}
```

```
};

static void gps_advance_locked(void) {
    if (fixed) return; /* RSU: nao se desloca */
    /*
    if (time_before(jiffies, last_change + GPS_MOVE_INTERVAL))
        return; /* ainda no intervalo */
    */
    quadrant = gps_neighbors[quadrant][get_random_u32() % 2];
    last_change = jiffies;
}
```

O movimento simulado satisfaz o requisito de “*deslocamento com velocidade variável*” em direção: a magnitude do passo (um quadrante adjacente) e a cadência mínima (3s) são fixas, mas a *direção* do vetor de deslocamento é sorteada a cada passo, gerando trajetórias não-determinísticas (e.g., um veículo pode oscilar entre dois quadrantes ou percorrer os quatro). Saltos diagonais (de Q0 para Q3 ou de Q1 para Q2) são proibidos pela tabela — um deslocamento físico realista cruza fronteiras compartilhadas.

2.4 Operação atômica e wrapper de *userspace*

O estado do módulo (`quadrant`, `last_change`, `fixed`) é compartilhado entre *todos* os processos da VM e protegido por um `DEFINE_MUTEX(gps_lock)`. O wrapper de *userspace* `GPS` (`src/network/gps.cpp`) encapsula o `open/ioctl/close` e usa o sentinela `QUADRANT_NONE = 0xFF` para sinalizar que o módulo não está disponível:

```
GPS::GPS() : _fd(::open("/dev/gps", O_RDWR)) { }

uint8_t GPS::quadrant() {
    if (_fd < 0) return QUADRANT_NONE;
    int q = 0;
    if (::ioctl(_fd, GPS_IOC_GET_QUADRANT, &q) < 0) return QUADRANT_NONE;
    return static_cast<uint8_t>(q);
}

void GPS::set_fixed(bool fixed) {
    if (!fixed || _fd < 0) return; /* default do modulo: se desloca */
    ::ioctl(_fd, GPS_IOC_SET_FIXED);
}
```

3 Etiqueta espacial das mensagens

3.1 Quadrante no Header do Protocol

A coordenada espacial da origem viaja como um campo do `Protocol::Header`, ao lado do timestamp e das portas. Esse campo é preenchido na NIC, no momento do envio, com a posição corrente da VM (obtida via `GPS::quadrant()`):

```
class Header {
public:
```

```

    int64_t    timestamp() const { return _timestamp; }
    Port       src_port()  const { return _src_port;  }
    Port       dst_port()  const { return _dst_port;  }
    PacketKind kind()      const { return _kind;      }
    uint8_t    quadrant()  const { return _quadrant;  } // <-- Etapa 4
    /* setters... */
protected:
    int64_t    _timestamp;
    Port       _src_port;
    Port       _dst_port;
    uint8_t    _quadrant;    /* Etapa 4: coordenada espacial da origem */
    PacketKind _kind;
};

```

Optou-se por colocar o quadrante no Header do Protocol — não no cabeçalho Ethernet — porque ele faz parte conceitualmente da identificação *do emissor*, exatamente como o timestamp e a porta lógica.

3.2 Etiqueta na Message (API)

A classe Message expõe a coordenada espacial como parte de um objeto composto Origin, refletindo diretamente o formato do enunciado $M = \{origin, timestamp, payload\}$: a *origem* agrupa o endereço físico, a porta lógica e o quadrante.

```

class Message {
public:
    static const uint8_t QUADRANT_NONE = 0xFF;

    // Etapa 4: origem composta = endereco fisico + porta + quadrante.
    class Origin {
    public:
        Vehicle_Protocol::Port    port() const { return address.port(); }
    }
    Vehicle_Protocol::Physical_Address physical_address() const {
        return address.paddr();
    }
    Vehicle_Protocol::Address address;
    uint8_t    quadrant;
};

const Origin & origin()    const { return _origin;          }
uint8_t    quadrant()    const { return _origin.quadrant; }
int64_t    timestamp()    const { return _timestamp;        }

private:
    Origin    _origin;    /* endereco + quadrante (Etapa 4) */
    int64_t    _timestamp; /* Etapa 3 */
    unsigned char _payload[MAX_SIZE];
    unsigned int _size;
    template <typename Channel> friend class Communicator;
};

```

A escrita dos campos da `Origin` é restrita ao `Communicator`, declarado `friend`; a aplicação só tem acesso de leitura via `m.origin()` (retorna `const Origin&`) ou diretamente via `m.quadrant()`. Isso satisfaz o requisito de “*mensagens etiquetadas com coordenadas espaciais na origem*”: a coordenada chega na `Message` já populada pela pilha, sem que o componente precise consultar o GPS ou conhecer detalhes do cabeçalho do protocolo.

4 Integração com a NIC: carimbo no envio e filtragem na recepção

4.1 Visão geral

A NIC é a camada responsável por (i) carimbar o quadrante de origem em cada envio e (ii) descartar pacotes vindos de outro quadrante na recepção. Para que isso seja possível sem que a NIC dependa do tipo concreto `Protocol::Header` (o que criaria dependência circular: o `Protocol` é *template* parametrizado pela NIC), adotamos o mesmo padrão de desacoplamento usado pelo SPTP da Etapa 3: injetam-se na NIC dois ponteiros para funções estáticas do `Protocol`, que sabem ler e escrever o campo `_quadrant` no `Header`.

4.2 Acessores estáticos e injeção na NIC

O `Protocol` expõe dois métodos estáticos que operam sobre o *payload* cru (o `Header` é classe aninhada, então o `Protocol` tem acesso direto ao campo privado):

```
static uint8_t read_quadrant(const void* payload) {
    return reinterpret_cast<const Header*>(payload)->quadrant();
}
static void write_quadrant(void* payload, uint8_t q) {
    reinterpret_cast<Header*>(payload)->quadrant(q);
}
```

No construtor do `Protocol`, esses dois ponteiros são injetados na `RawSocketNIC` (a NIC inter-VM):

```
if (SharedMemoryNIC::is_gateway_process()) {
    _socket_nic = new RawSocketNIC();
    _socket_nic->attach(this, PROTO);
    _socket_nic->setup_read_quadrant (&read_quadrant);
    _socket_nic->setup_write_quadrant(&write_quadrant);
}
```

A NIC armazena-os como ponteiros tipados e os utiliza tanto no envio (para escrever o quadrante no `Header`) quanto na recepção (para lê-lo antes de decidir descarte). A `SharedMemoryNIC` mantém os ponteiros nos seus valores *default* (*no-op* para escrita, `QUADRANT_NONE` para leitura), o que garante a propriedade do próximo item: tráfego intra-veículo não sofre filtragem espacial.

4.3 Carimbo no envio

Na `NIC::send`, o quadrante corrente da VM (consultado ao módulo via `GPS::quadrant()`, encapsulado em `Engine::engine_current_quadrant()`) é gravado no `Header` do pacote logo antes de chamar `Engine::engine_send`:

```

int send(Buffer<Ethernet::Frame> *buf) {
    _write_quadrant(buf->data()->payload(),
                    Engine::engine_current_quadrant());
    int bytes = Engine::engine_send(buf->data(),
                                    Ethernet::HEADER_SIZE + buf->size());

    /* ... estatísticas ... */
}

```

4.4 Filtragem na recepção

Quando um frame chega à RawSocketEngine, a NIC lê o quadrante da origem (via o acessor injetado) e o passa para `engine_should_drop_frame`, que o compara com o quadrante local e descarta a mensagem se forem distintos:

```

/* dentro do callback de recepcao do NIC */
uint8_t q = _read_quadrant(buf->data()->payload());
if (Engine::engine_should_drop_frame(*frame, _address, q)) {
    free(buf); /* drop: outro MAC ou outro quadrante */
    return;
}

/* RawSocketEngine */
bool engine_should_drop_frame(const Ethernet::Frame & frame,
                              const Ethernet::Address & local,
                              const uint8_t & origin_q) const {
    if (frame.src() == local) return true; /* self-drop */
    uint8_t my_q = _gps.quadrant();
    if (my_q != GPS::QUADRANT_NONE &&
        origin_q != GPS::QUADRANT_NONE &&
        my_q != origin_q) return true; /* cross-quadrant */
    /*
    return false;
}

```

A condição com `QUADRANT_NONE` preserva a compatibilidade com cenários em que o módulo GPS não está carregado (testes anteriores): se algum dos lados não tem GPS, não há filtragem espacial.

4.5 Recepção na aplicação: `Protocol::receive` e `Communicator`

Para que o quadrante chegue à camada de aplicação dentro da `Message`, `Protocol::receive` foi estendido para extrair o campo do `Header` (devolvido por parâmetro de saída) e o `Communicator` popula os dois campos da `Origin`: `address` (vindo do MAC + porta) e `quadrant` (vindo do `Header`).

```

/* Protocol::receive */
Packet *packet = reinterpret_cast<Packet *>(buf->data()->payload());
if (from)      *from = Address(buf->data()->src(), packet->src_port());
if (ts)        *ts      = packet->timestamp();
*quadrant      = packet->quadrant(); /* <-- Etapa 4 */

```



```

/* Communicator::receive */
uint8_t q = Message::QUADRANT_NONE;    /* default seguro se receive falhar */
int size = _channel->receive(buf, &from, &ts, &q,
                             message->data(), message->size());
message->_size = size;
message->_origin.address = from;         /* endereço físico + porta lógica */
message->_timestamp = ts;
message->_origin.quadrant = q;           /* coordenada espacial (Etapa 4) */

```

A propagação preserva o quadrante real durante o encaminhamento gateway → componente (que é feito por `memcpy` do frame inteiro, incluindo o `Header`), de modo que a aplicação recebe exatamente o quadrante carimbado pelo emissor.

5 Compartilhamento intra-veículo

O requisito de que “*componentes de um mesmo sistema autônomo compartilham a percepção do espaço e concordam com exatidão sobre sua localização*” é atendido por dois mecanismos complementares:

- **Estado global ao módulo de kernel.** O módulo `gps.ko` mantém um único estado de quadrante por carregamento. Como o estado é compartilhado por todos os processos da VM e protegido por `gps_lock`, qualquer componente que abrir `/dev/gps` obtém *exatamente o mesmo valor* retornado a qualquer outro componente da mesma VM no mesmo instante. Não há como dois componentes do mesmo sistema discordarem sobre o quadrante.
- **Sem filtragem espacial no canal intra-VM.** O canal de memória compartilhada (`SharedMemoryEngine`) não sofre filtragem por quadrante: `engine_current_quadrant()` retorna `QUADRANT_NONE` no SHM, e `engine_should_drop_frame` sempre retorna `false`. Componentes do mesmo sistema autônomo, portanto, sempre se comunicam, independentemente da posição (que aliás é a mesma para todos eles).

6 Validação experimental

A validação da Etapa 4 é feita pelo teste `quadrant` (`tests/quadrant.cpp`), executado pelo alvo `test` do `make`. O cenário envolve nove VMs em QEMU, sobre o mesmo kernel `PREEMPT_RT 6.15.5` das etapas anteriores, agora com `gps.ko` carregado no `init`:

- **VM1–VM4 (RSUs).** Carregam o módulo com `initial_quadrant = {0, 1, 2, 3}` e chamam `GPS::set_fixed(true)` para travar a posição (e em seguida executam a classe `RSU`, que opera como mestre `SPTP`). Antes de subir o `SPTP`, o teste amostra o GPS durante 15 s (cinco intervalos de 3 s) e *aborta* se o quadrante mudar — prova diretamente que `SET_FIXED` congela o estado conforme o contrato.
- **VM5–VM9 (veículos).** Carregam o módulo sem parâmetro (posição inicial aleatória) e executam dois componentes: `QSender`, que transmite em *broadcast* a cada 1 s; e `QReceiver`, que recebe, classifica cada mensagem como *same-quadrant* ou *cross-quadrant* (comparando `m.quadrant()` com o quadrante local) e valida as assertivas finais.

As assertivas validadas pelos veículos são duas:

1. **Liveness:** `received >= 8` — houve troca real de mensagens.
2. **Filtragem espacial:** `cross_quadrant <= received / 10` — mensagens vindas de outro quadrante praticamente não chegam (um filtro *quebrado* entregaria $\sim 75\%$ de cross, dado que três dos quatro quadrantes são “errados”).

7 Resultados

A Tabela 1 apresenta o resultado de um *run* representativo do cenário **quadrant** (90 s de simulação, 1 s por envio).

Tabela 1: Cenário **quadrant**: 9 VMs (vm1–vm4 RSUs ancoradas, vm5–vm9 veículos móveis), simulação de 90 s. *Cross-quadrant* é a contagem (na aplicação) de mensagens classificadas como vindas de outro quadrante; *quad_changes* é o número de transições observadas na linha do tempo de cada VM.

VM (papel)	Recebidas	Same-quad.	Cross-quad.	Mudanças	Quadrante
vm1 (RSU)	—	—	—	0	fixo em 0
vm2 (RSU)	—	—	—	0	fixo em 1
vm3 (RSU)	—	—	—	0	fixo em 2
vm4 (RSU)	—	—	—	0	fixo em 3
vm5 (veículo)	27	27	0	30	—
vm6 (veículo)	74	74	0	30	—
vm7 (veículo)	39	39	0	30	—
vm8 (veículo)	74	74	0	30	—
vm9 (veículo)	89	89	0	30	—

Análise dos resultados:

- **RSUs permaneceram fixas.** As quatro RSUs registraram zero transições durante toda a simulação, mesmo amostrando o GPS por janelas de 15 s que cruzam cinco intervalos de avanço potencial — confirma que `GPS_IOC_SET_FIXED` congela o estado.
- **Veículos se deslocaram.** Cada veículo realizou 30 transições em 90 s (uma a cada 3 s, como esperado pela cadência do avanço preguiçoso), seguindo a tabela de vizinhança — nenhum salto diagonal foi observado nos logs do `QSender`.
- **Filtragem espacial 100%.** Todas as mensagens entregues pela NIC à aplicação foram classificadas como *same-quadrant*; `cross_quadrant` foi zero em todos os veículos. Em runs com cadência mais alta de envio (500 ms em vez de 1 s) observa-se ocasionalmente `cross_quadrant = 1`, atribuível à re-amostragem do quadrante local pela aplicação após a NIC já ter decidido entregar — artefato de *timing* na fronteira entre intervalos de 3 s, sem implicação sobre a correção da filtragem propriamente dita, sempre dentro do limiar `cross ≤ received/10`.

- **Etiquetagem das mensagens.** O fato de *same_quadrant* ser sempre igual a *received* demonstra que a *Message* chega à aplicação com o quadrante de origem corretamente populado, e que esse valor coincide com o quadrante do receptor no momento da entrega — requisito de etiquetagem espacial da etapa atendido por construção.
- **Compartilhamento intra-VM.** O *QSender* e o *QReceiver* de cada veículo, processos distintos dentro da mesma VM, sempre amostraram o mesmo quadrante via */dev/gps* — propriedade que decorre diretamente do estado global ao módulo, sem ajuste algum no código do componente.

8 Conclusão

A Etapa 4 acrescentou a dimensão espacial ao sistema, cumprindo os cinco requisitos do enunciado:

1. **Etiquetagem espacial.** *Message* passou a expor um objeto composto *Origin* (endereço + porta + quadrante), casando diretamente com o formato $M = \{origin, timestamp, payload\}$ do enunciado. A escrita dos campos é restrita ao *Communicator* via *friend*, e o quadrante é preenchido a partir do valor carimbado no *Protocol::Header* pela NIC no momento do envio.
2. **Precisão por quadrante.** A grade 2×2 de quadrantes (Q0–Q3) é exposta pelo módulo de kernel e usada como única unidade de identificação espacial.
3. **Componentes compartilham a percepção.** O estado é global ao módulo *gps.ko* e o canal intra-VM (SHM) não filtra por quadrante — dois componentes do mesmo sistema sempre veem o mesmo valor e sempre se comunicam.
4. **Módulo de kernel com interface de leitura.** A interface é */dev/gps* via *ioctl(GPS_IOC_GET_QUADRANT)*, implementada como *misc device*.
5. **Inicialmente aleatório, depois deslocamento simulado.** A posição inicial é sorteada no *insmod*; em seguida, a cada ≥ 3 s, o quadrante avança para um vizinho sorteado da grade 2×2 — o passo (magnitude) é fixo, mas a direção do vetor de deslocamento é variável.

O teste *quadrant*, com 9 VMs em QEMU, confirma a implementação: as RSUs permanecem ancoradas (*quad_changes* = 0), os veículos se deslocam (*quad_changes* = 30) e a filtragem espacial elimina todas as mensagens cross-quadrante na entrega à aplicação. A propagação do quadrante até a *Message* é exercitada pelo casamento perfeito entre *same_quadrant* e *received*, provando que a aplicação realmente recebe mensagens etiquetadas com a coordenada espacial do emissor — o requisito central da etapa.